Evaluation functions



Zola Mahlaza

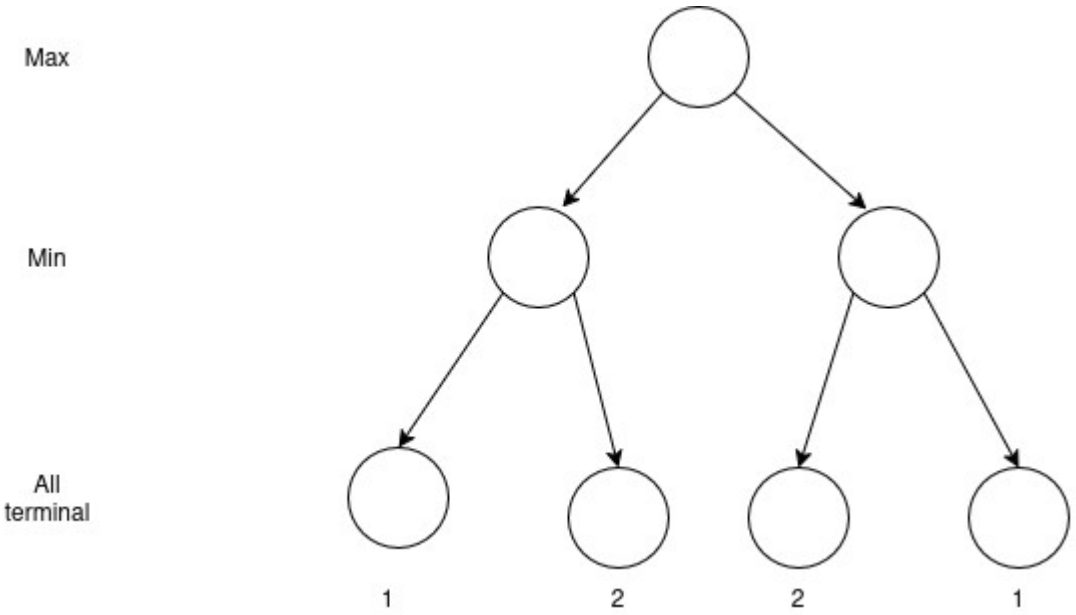
Department of Computer Science

Feb 2026

Alpha-beta pruning example

- Alpha-beta pruning example
- Evaluation functions

Alpha-beta example

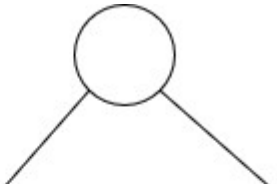


Alpha-beta example

Note: [value alpha beta]

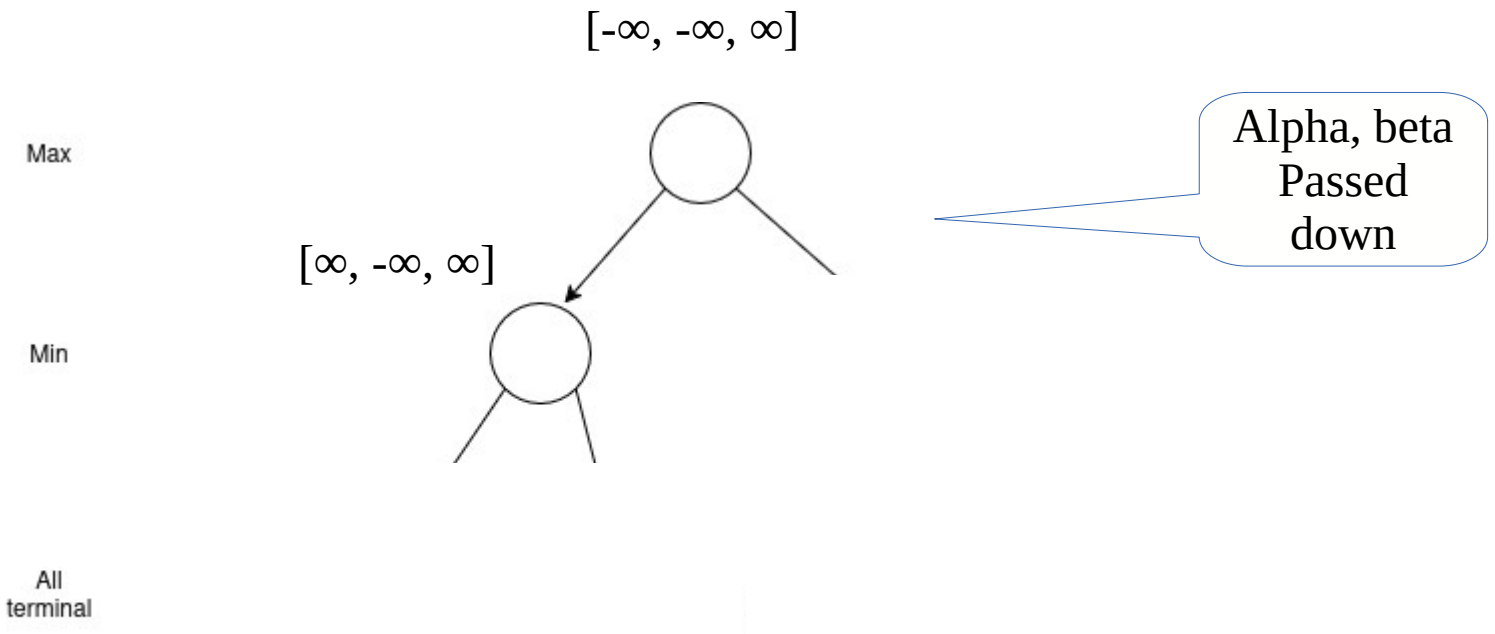
[-∞, -∞, ∞]

Max



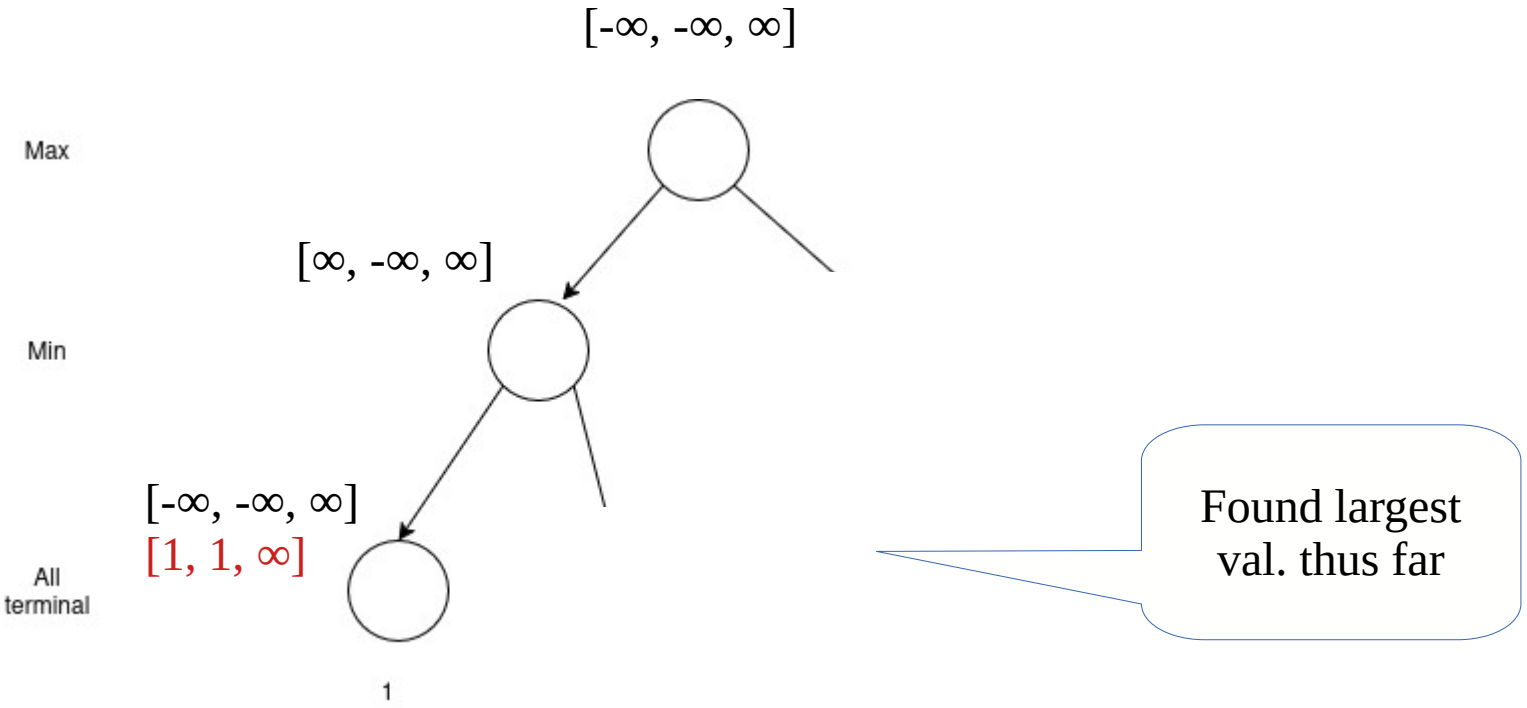
Alpha-beta example

Note: [value alpha beta]



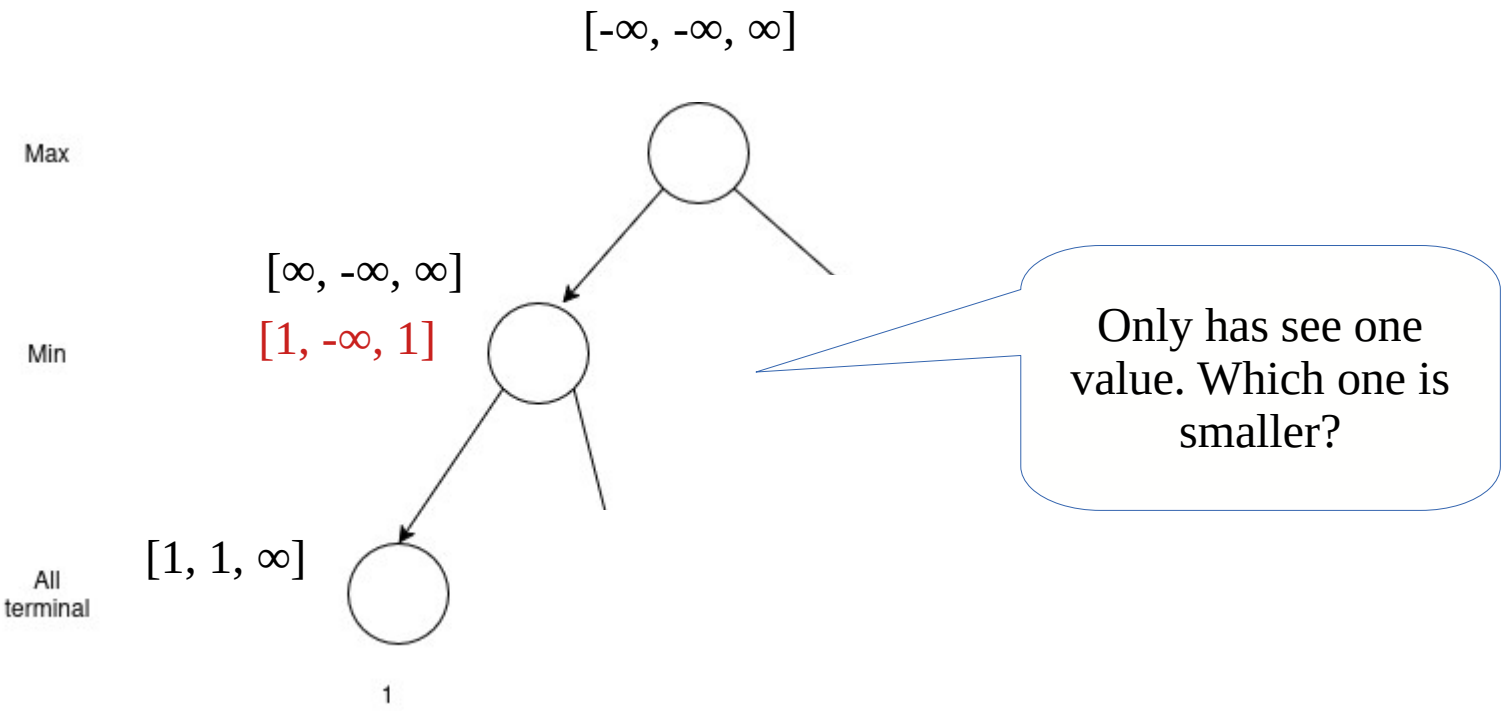
Alpha-beta example

Note: [value alpha beta]



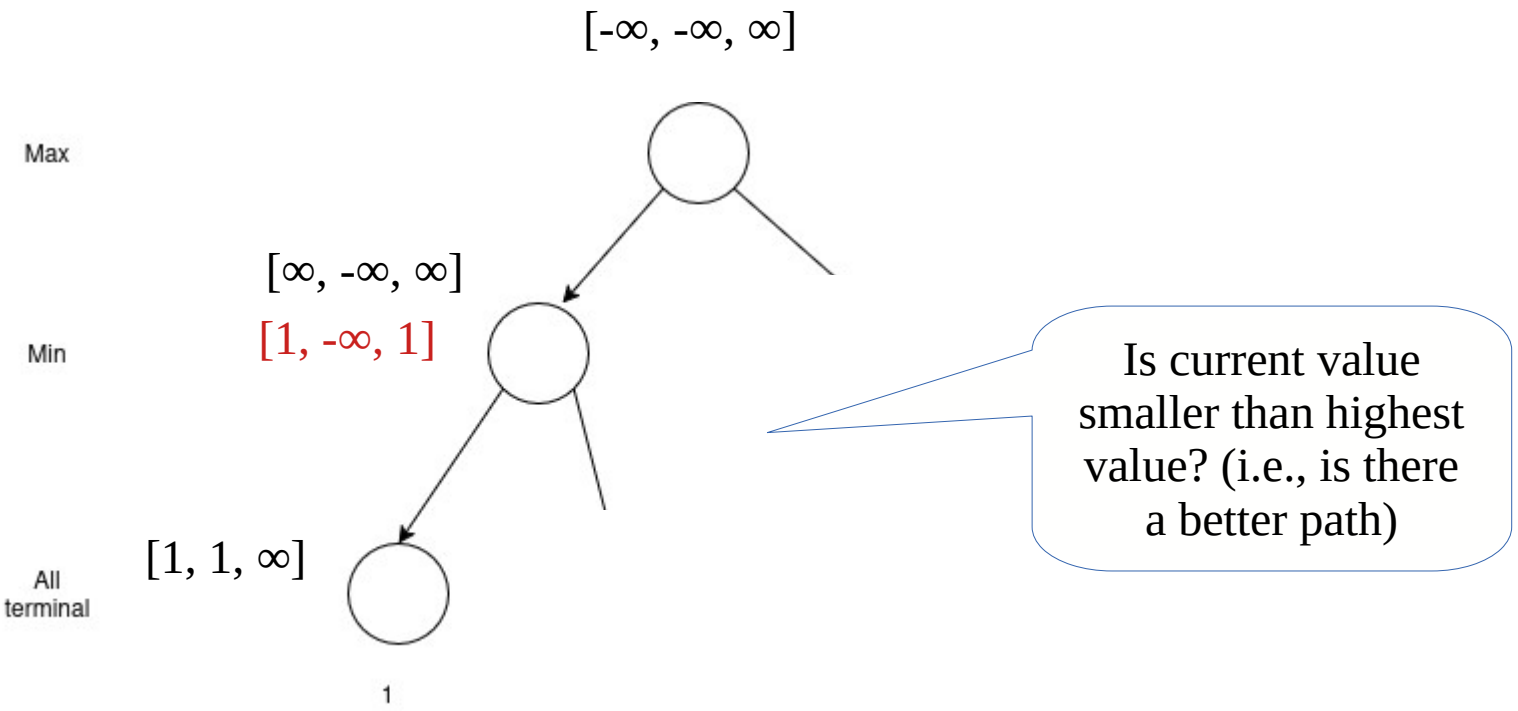
Alpha-beta example

Note: [value alpha beta]



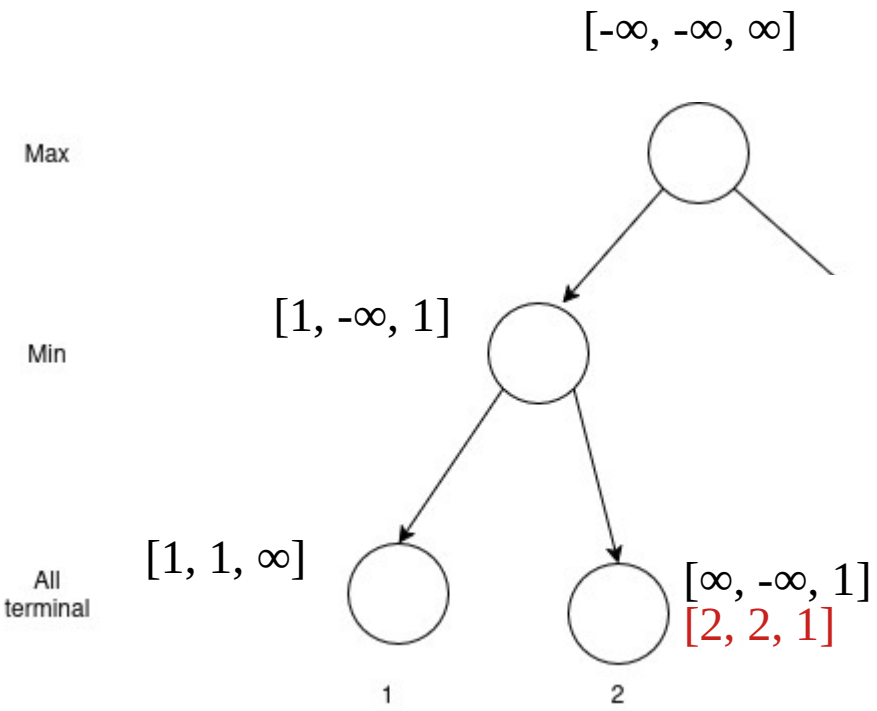
Alpha-beta example

Note: [value alpha beta]



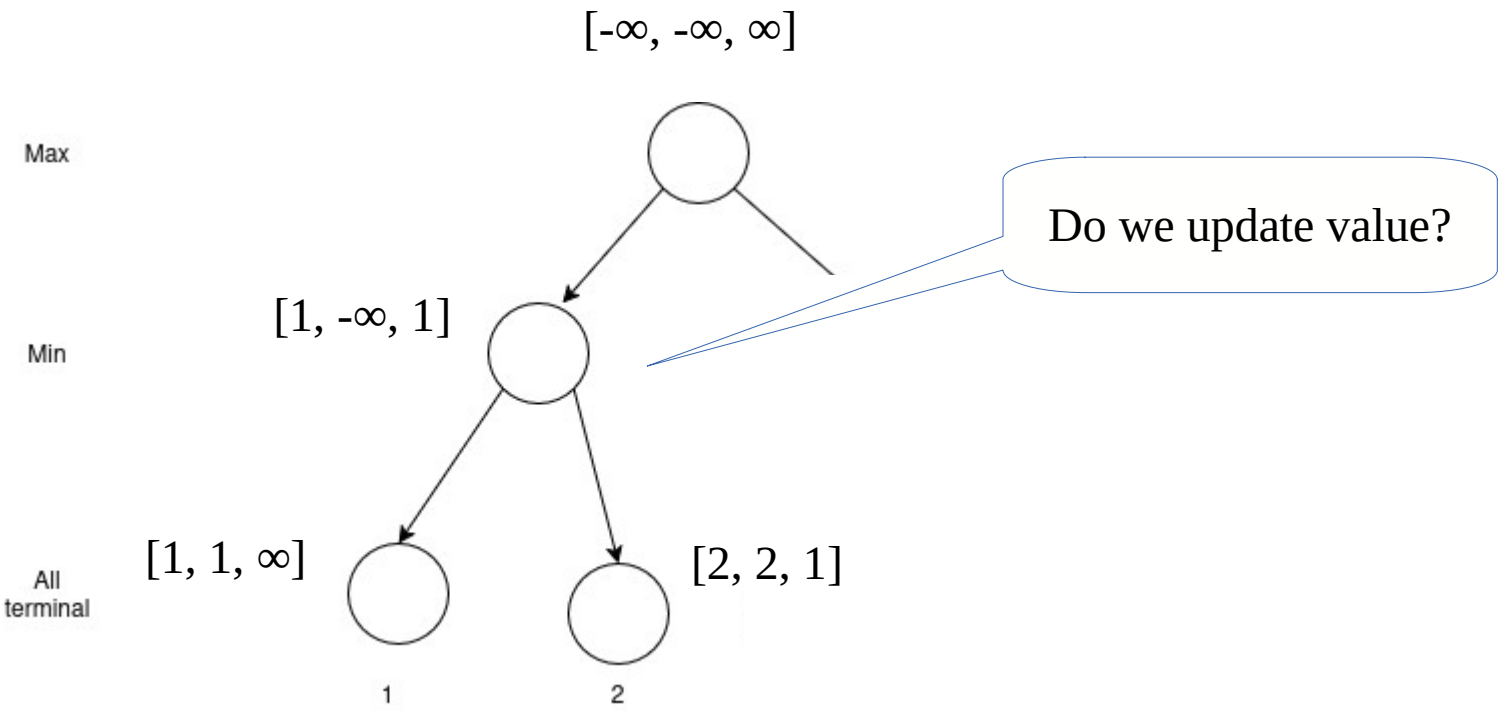
Alpha-beta example

Note: [value alpha beta]



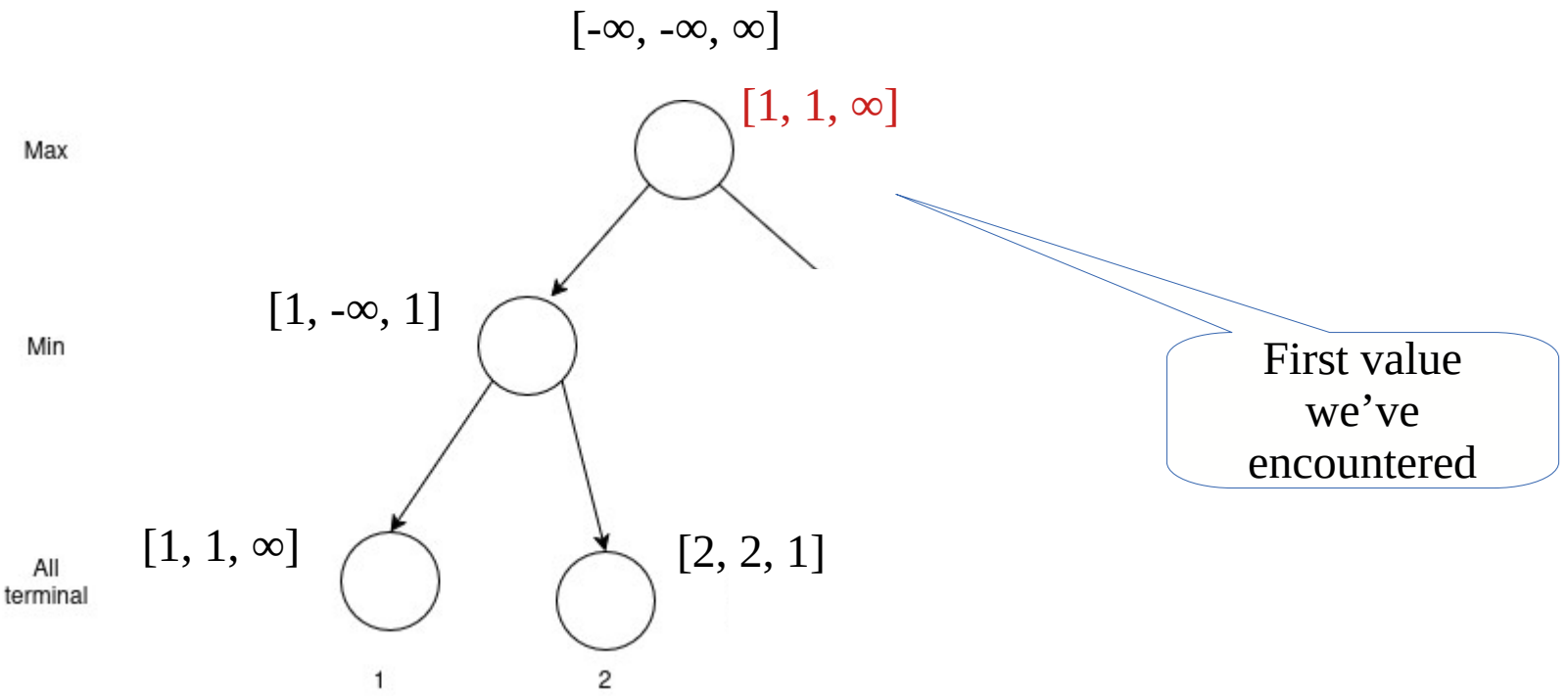
Alpha-beta example

Note: [value alpha beta]



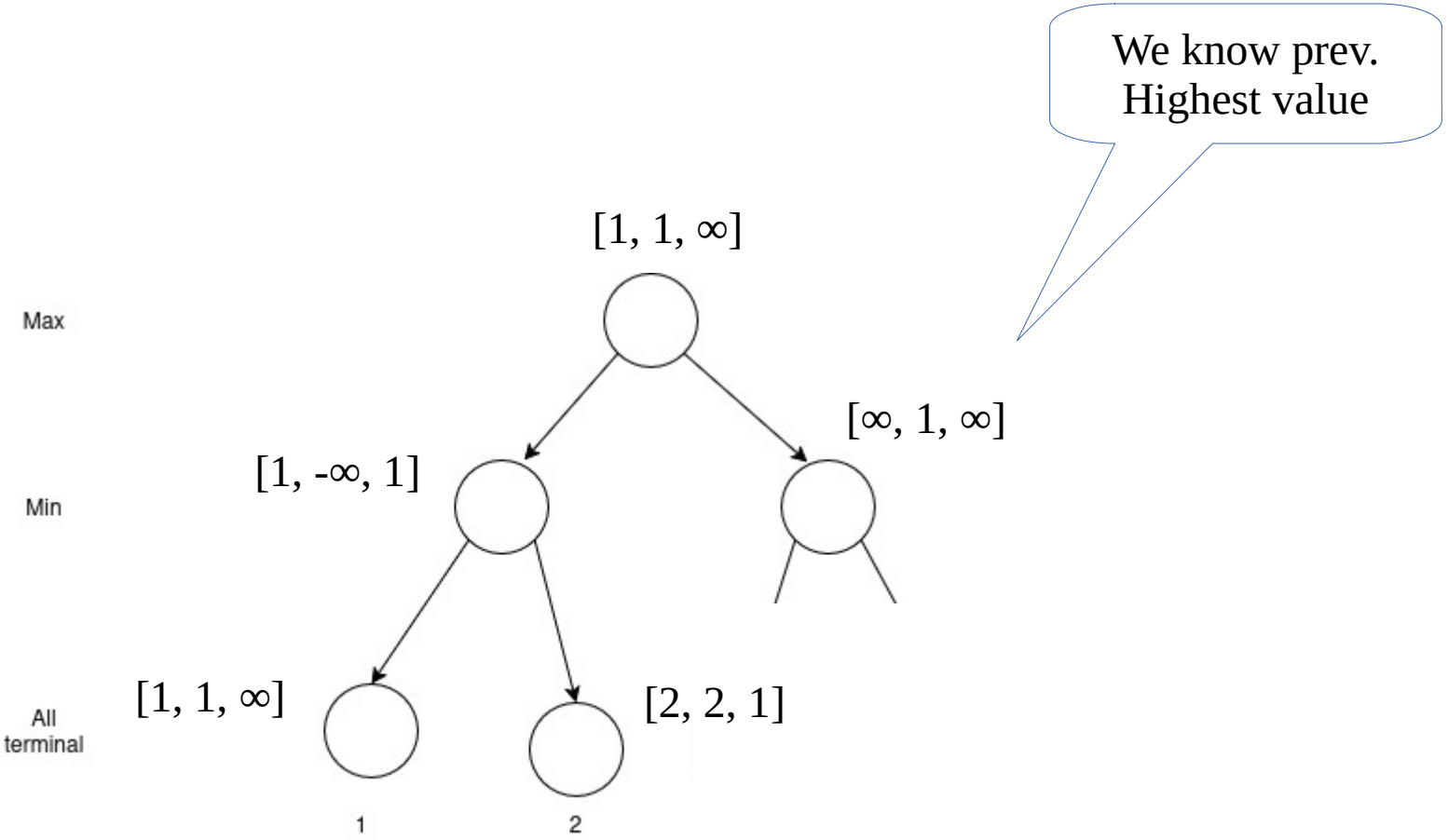
Alpha-beta example

Note: [value alpha beta]



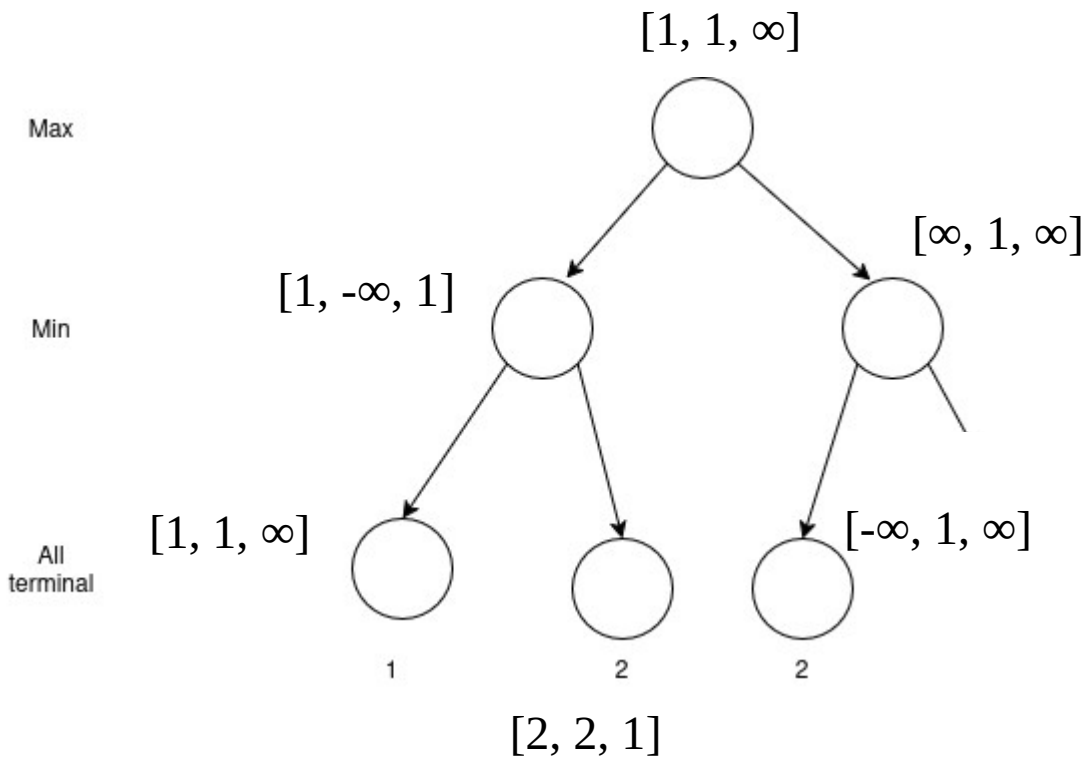
Alpha-beta example

Note: [value alpha beta]



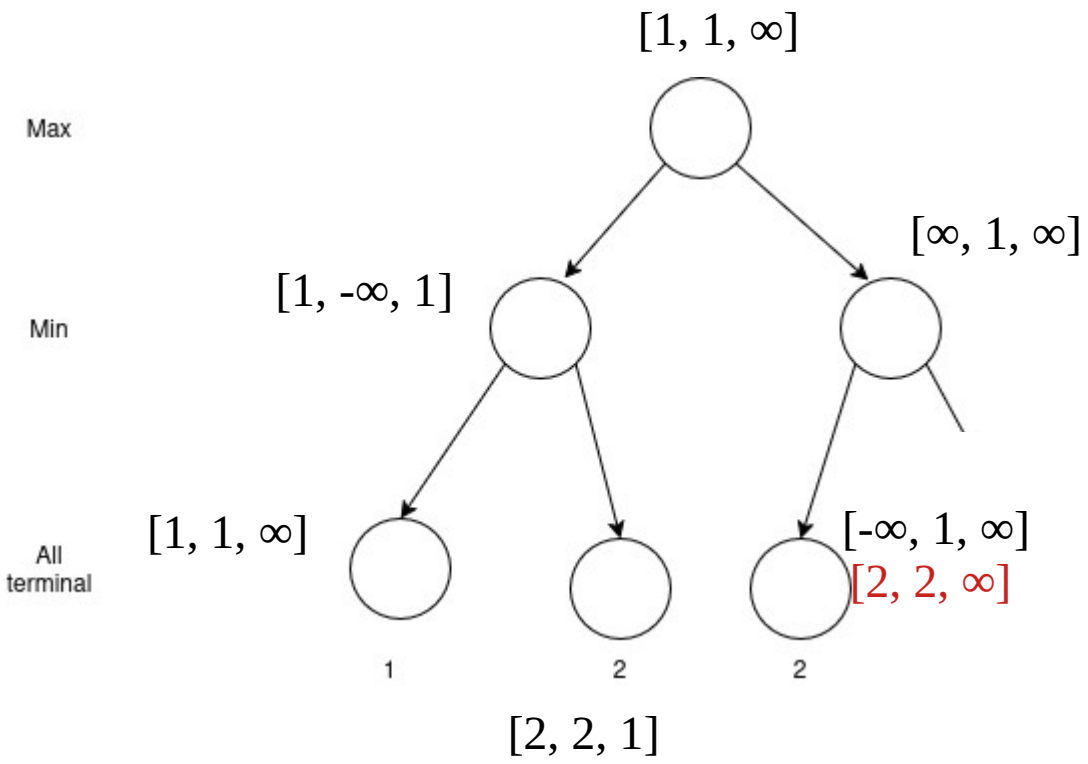
Alpha-beta example

Note: [value alpha beta]



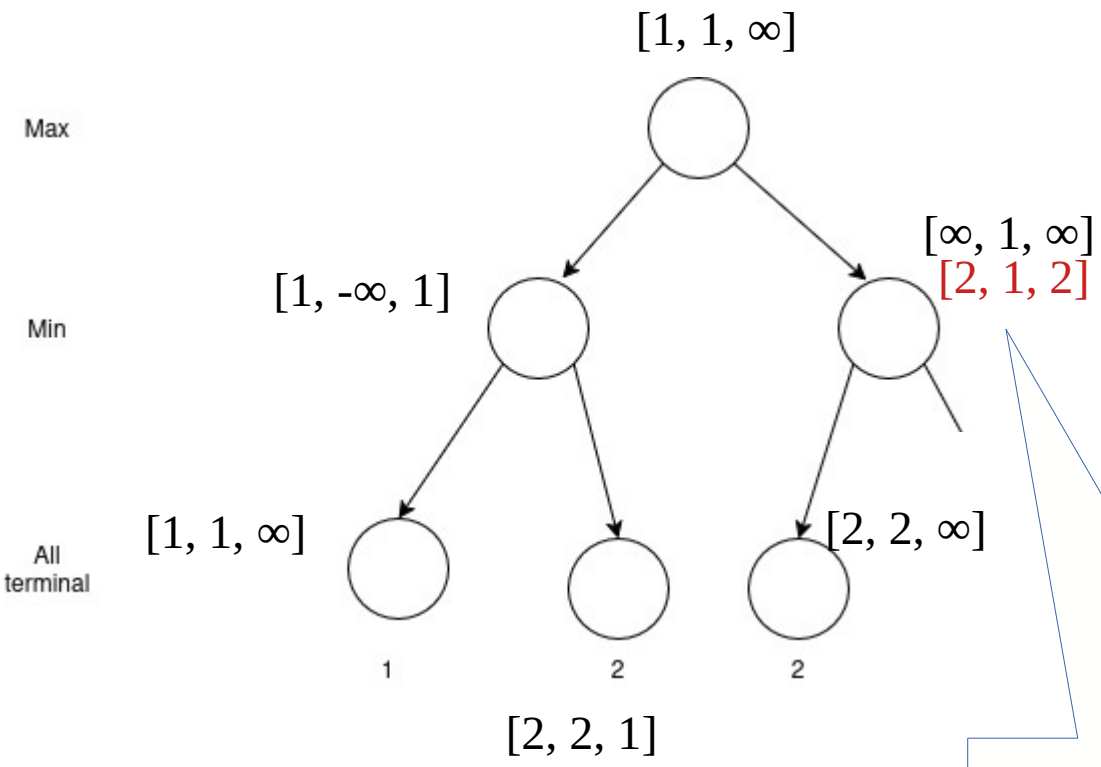
Alpha-beta example

Note: [value alpha beta]



Alpha-beta example

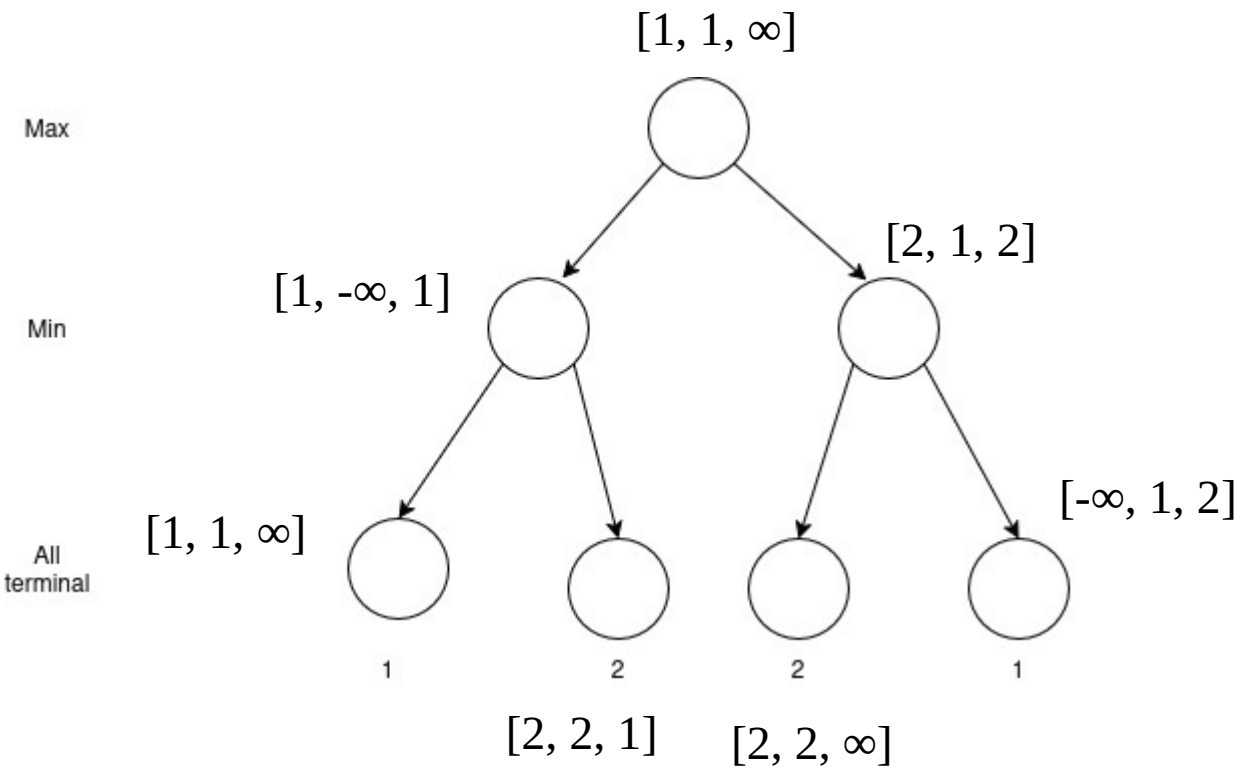
Note: [value alpha beta]



If 2 <= alpha then current branch wouldn't be considered so we could prune siblings

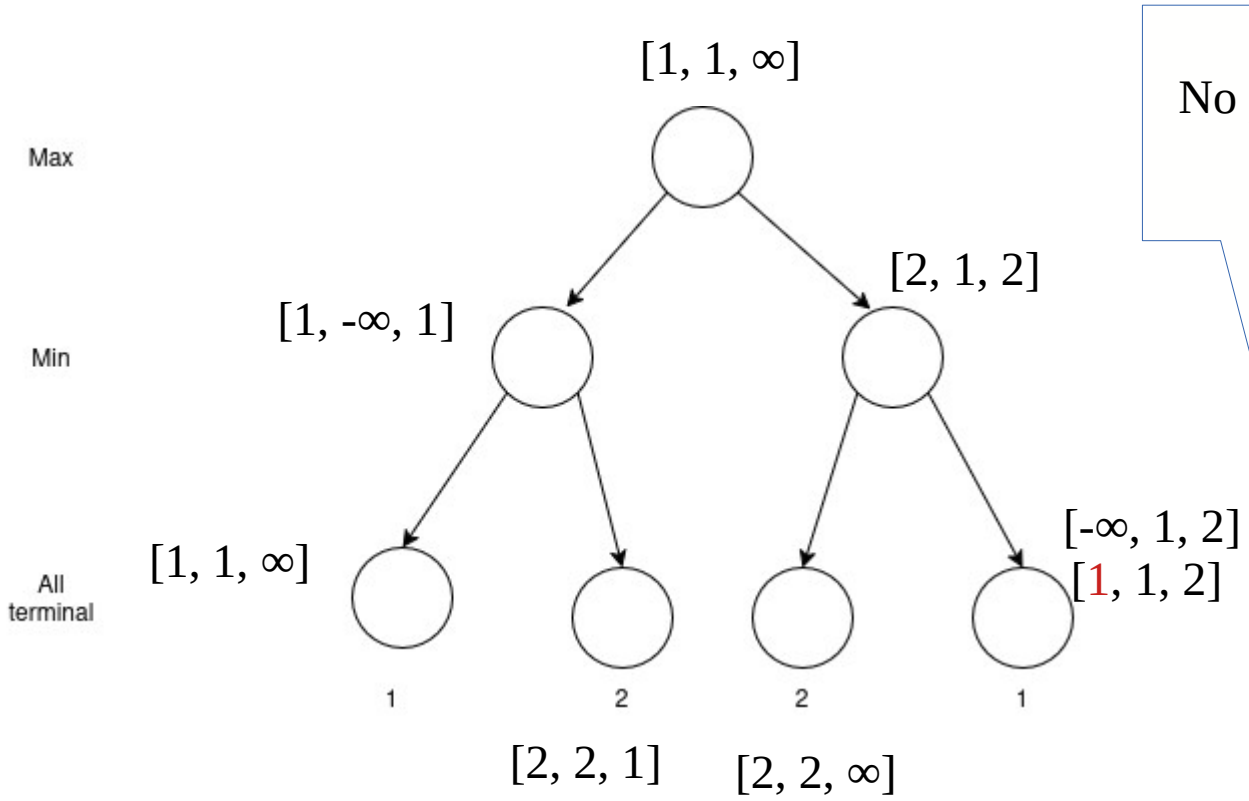
Alpha-beta example

Note: [value alpha beta]



Alpha-beta example

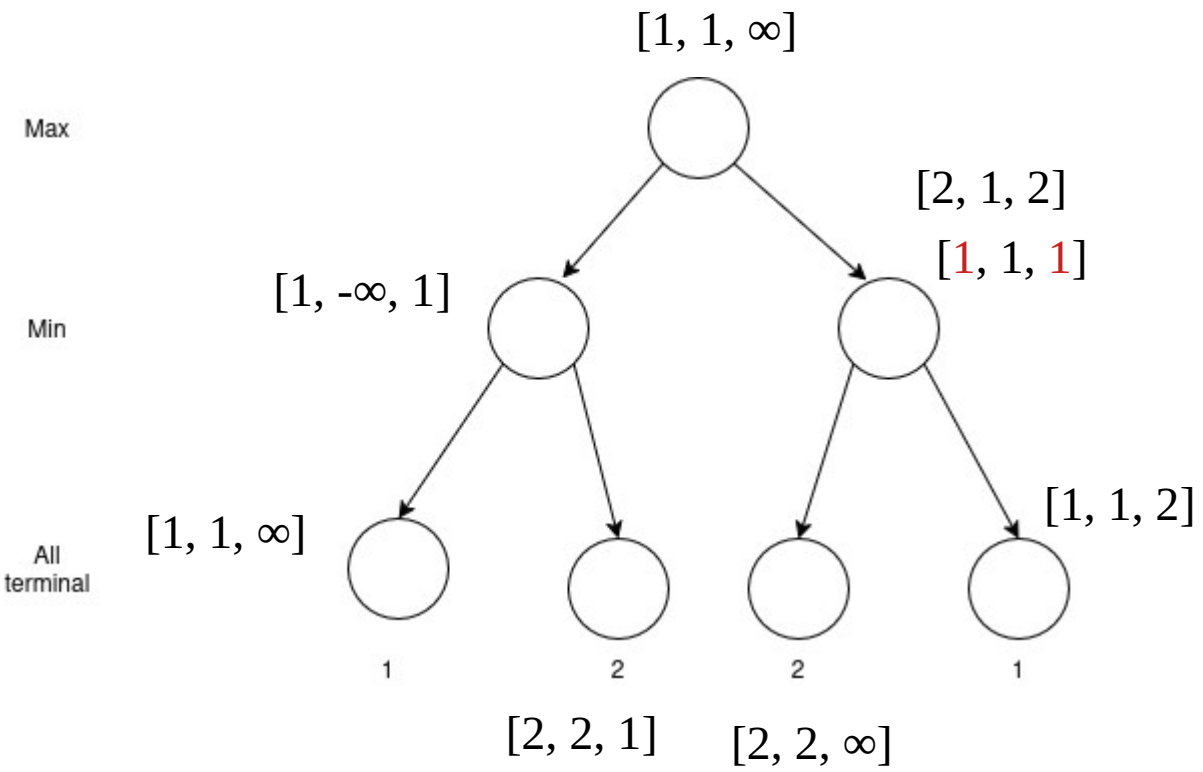
Note: [value alpha beta]



No point in updating Max value

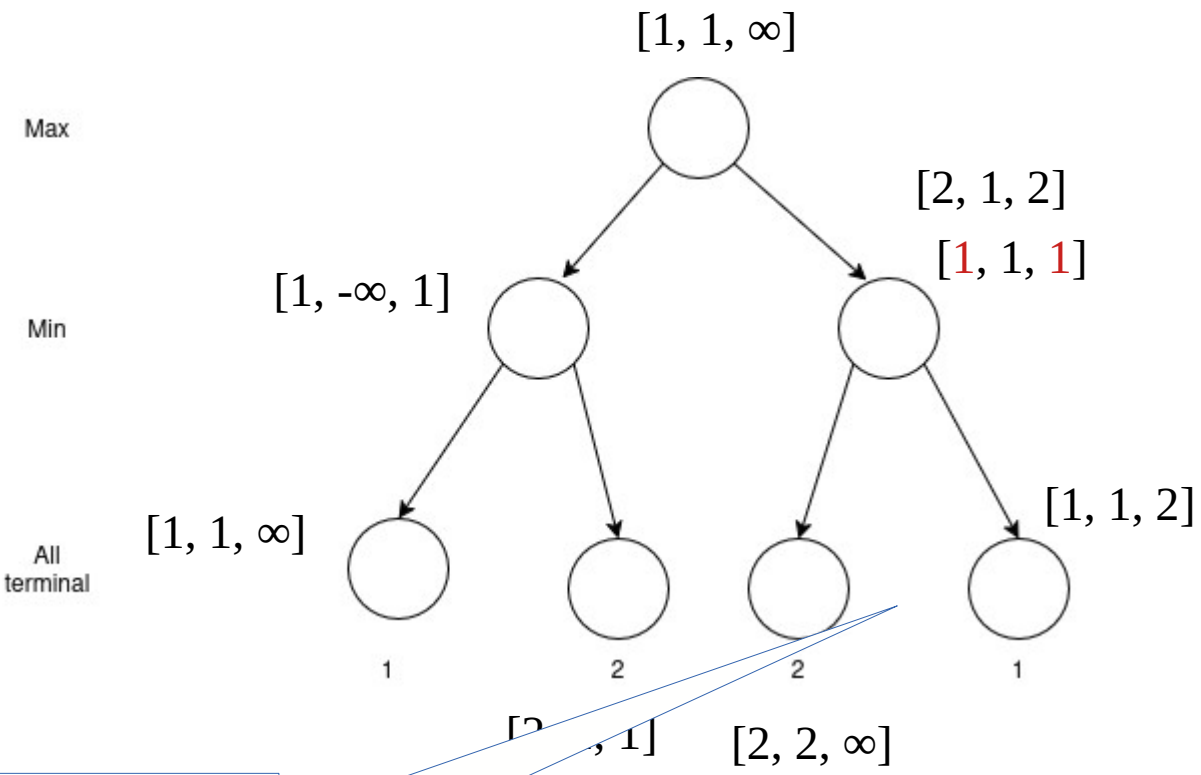
Alpha-beta example

Note: [value alpha beta]



Alpha-beta example

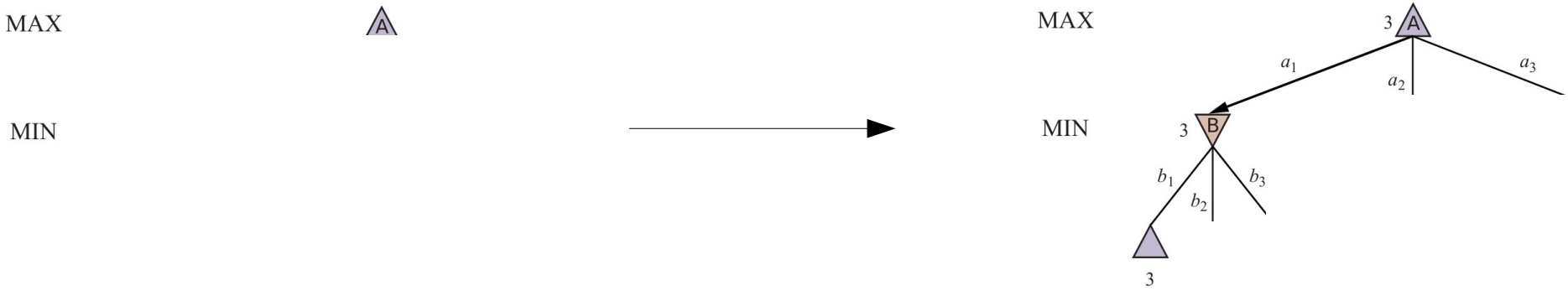
Note: [value alpha beta]



Perfect ordering of successor States is important

Utility function in minimax

- Key functions in minimax:
  - Is-terminal(s): Method for determining whether we are in terminal state
  - Utility(s, p): payoff function. Assigns numerical score to player in terminal state



Utility function in minimax

- What happens when the depth of game tree is too deep?
- Alpha-beta search is still exponential (improvement to $O(b^{m/2})$)
- Idea: Cut off the search early and apply a heuristic evaluation function to states
- Do not rely on utility function exclusively

Utility function in minimax

- Introduction of new functions:
  - Is-terminal(s): Method for determining whether we are in terminal state
  - IS-CUTOFF (s, d): Method for determining whether stop exploring states post a specific depth.
  - Utility(s, p): payoff function. Assigns numerical score to player in terminal state
  - Eval(s, p): estimate of pay off (distance to the goal) for player p. For terminal states, Eval(s, p) = Utility(s, p)

Utility function in minimax

- Introduction of new functions:
  - Is-terminal(s): Method for determining whether we are in terminal state
  - IS-CUTOFF (s, d): Method for determining whether stop exploring states post a specific depth.
  - Utility(s, p): payoff function. Assigns numerical score to player in terminal state
  - Eval(s, p): estimate of pay off (distance to the goal) for player p. For terminal states, Eval(s, p) = Utility(s, p)
- Eval must be admissible wrt wins (i.e., Eval(win, p) <= Utility(win, p))
- It must never underestimate losses (i.e., Utility(loss, p) < Eval(loss, p))

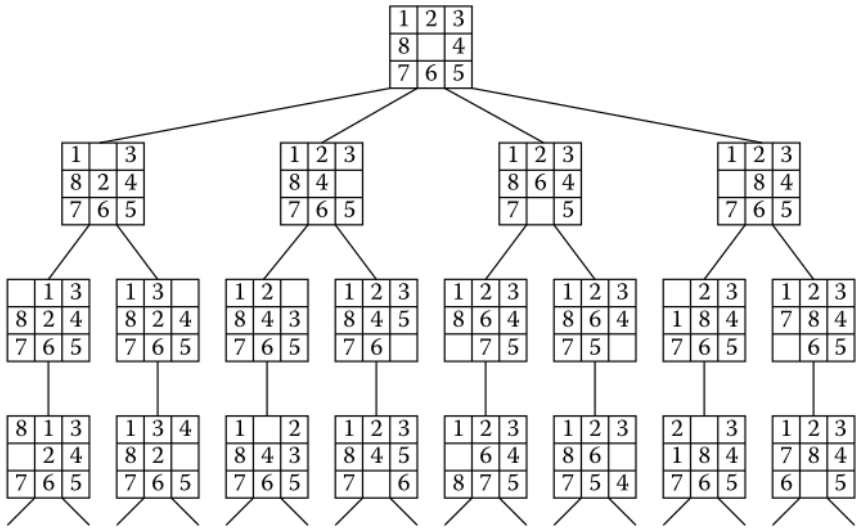
Utility function in minimax

- We can then use a heuristic to determine the minimax value of each state
  - No need to depth-first until terminal states
- Simple solution: IS-CUTOFF (state, depth) returns true for all depth >= chosen d (and terminal states)

$$\text{H-MINIMAX}(s,d) = \begin{cases} \text{EVAL}(s,\text{MAX}) & \text{if IS-CUTOFF}(s,d) \\ \max_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if TO-MOVE}(s) = \text{MIN}. \end{cases}$$

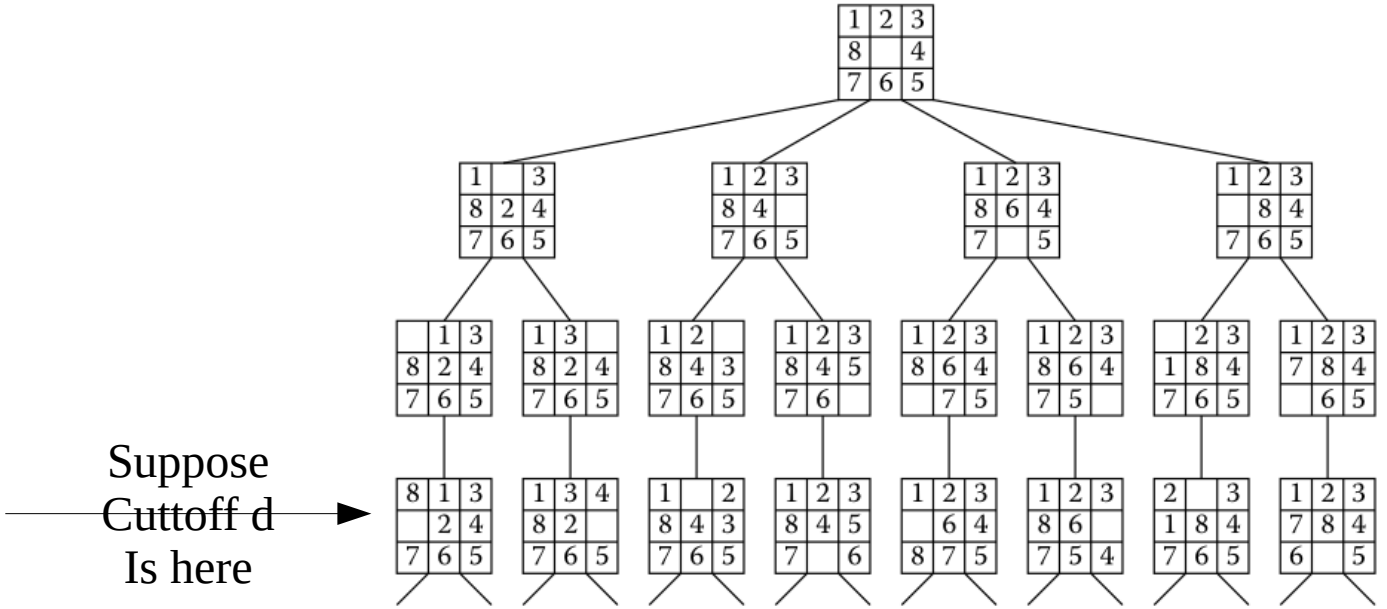
What makes a good EVAL

- Suppose you are at state s, the computation must not take too long
- Function should be strongly correlated with the actual chances of winning
  - When dealing with deterministic and perfect-information games, what is chance?



EVAL and chance

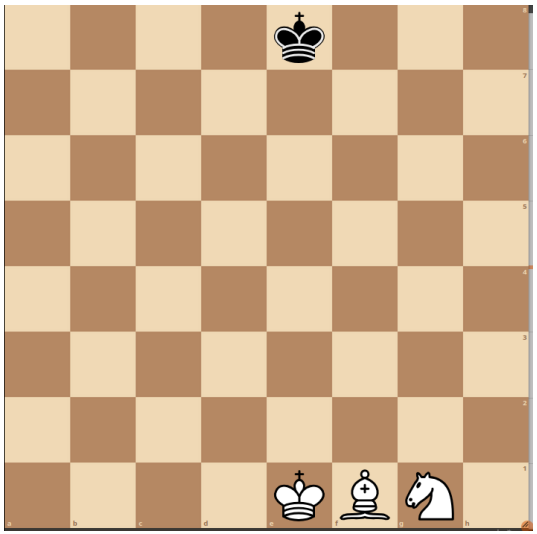
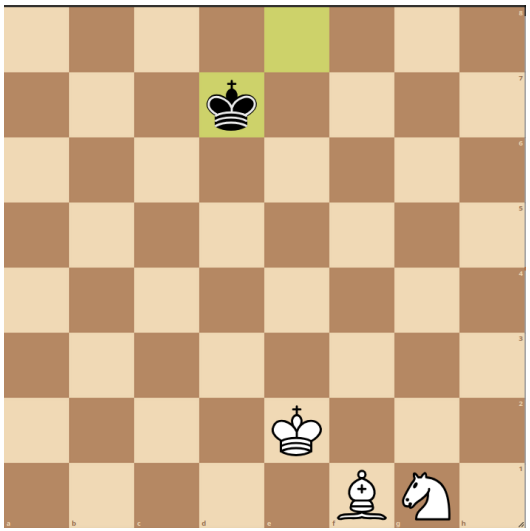
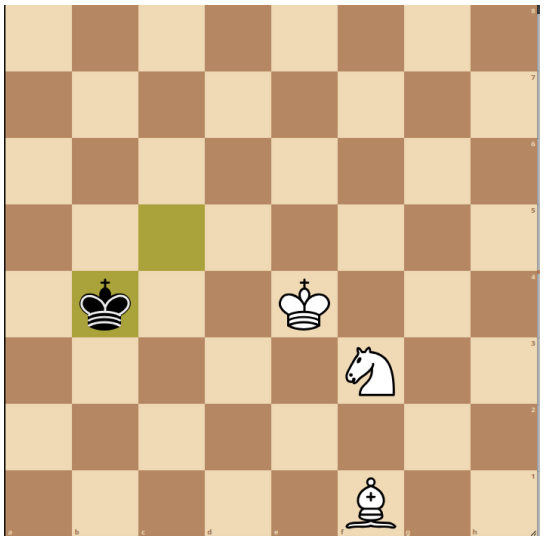
- Suppose you are at state s, the computation must not take too long
- Function should be strongly correlated with the actual chances of winning
  - When dealing with deterministic and perfect-information games, what is chance?



Uncertainty about future

EVAL and chance

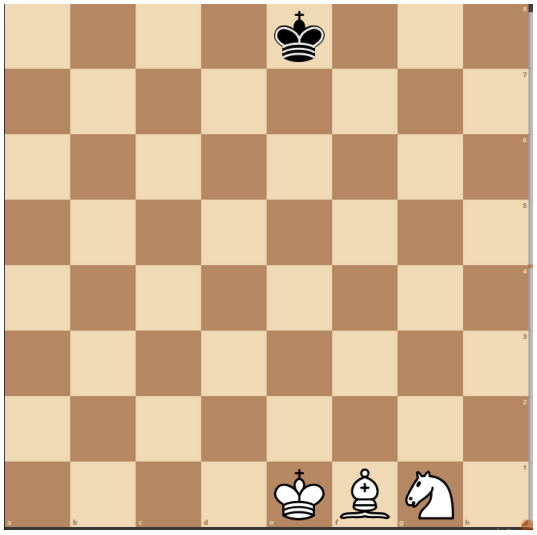
- Most EVAL functions compute value based on features of state
  - e.g., # of pawns, # knights, location of pawns, etc
- Features define categories of states
  - e.g., consider knight + bishop checkmate

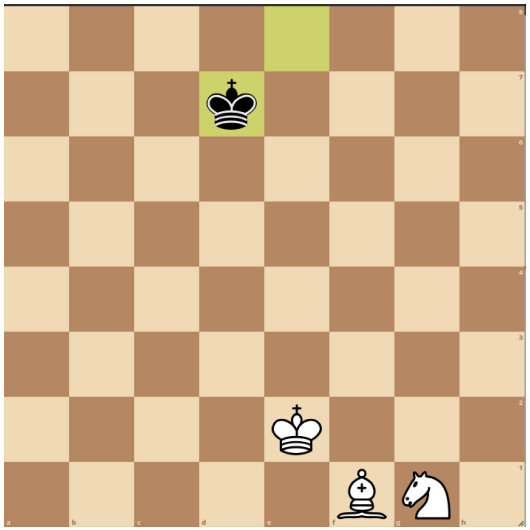




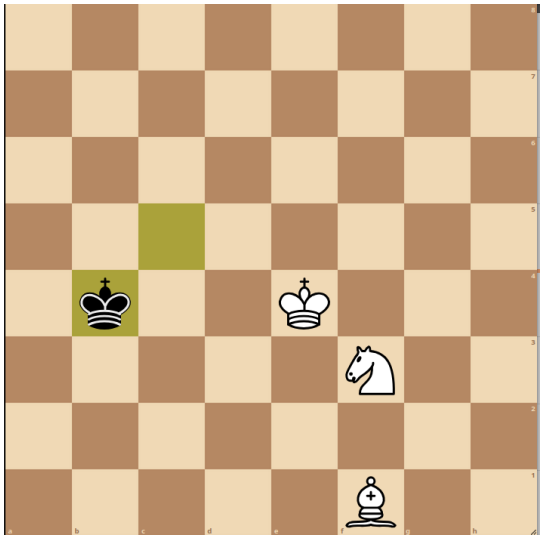
EVAL and chance

- Most EVAL functions compute value based on features of state
  - e.g., # of pawns, # knights, # pawns, etc
- Features define categories of states
  - e.g., consider knight + bishop checkmate

Different states but can be considered the Same endgame!

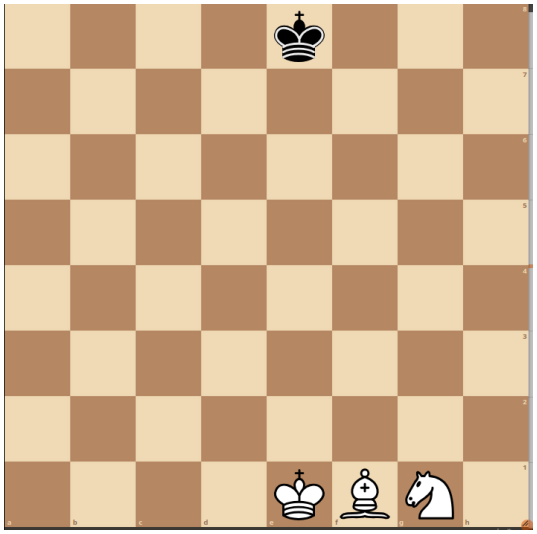
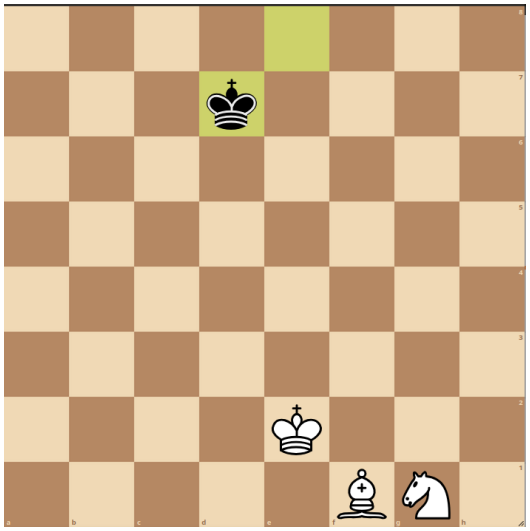
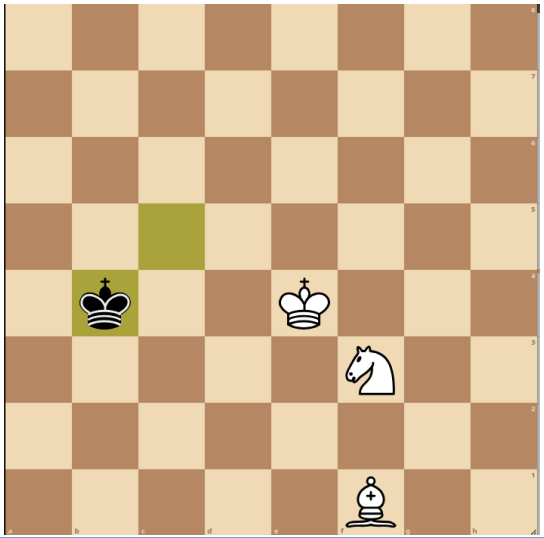






EVAL and chance

- Irrespective of state, we can determine chances of winning based on experience
  - E.g., prob(win) = 60%, prob(draw) = 40%, prob(loss) = 0%
- For all such states, expected value give you heuristic
  - prob(win)*1 + prob(draw)*0.5 + prob(loss)*0

Drawback of analysis for EVAL

- Practically, this is not scalable. Too many categories
- Also assumes historical data for prob. calculation
- Often, calculate separate feature numerical contributions + combine them
  - e.g., How likely are to win with x pawns? Etc.
- Treat all the features as independent
- EVAL then becomes a weighted combination:

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^{n} w_i f_i(s)$$

Implementation of EVAL in practice

- Evaluation function must be strongly correlated with winning
  - Better win prospects → higher evaluation values
  - Linearity is not important. Function must be monotone (i.e., EVAL(s) > EVAL (result(s, a))
  - Use nonlinear combinations of features (e.g., does # of pawns matter more than king satefy)
- It is often difficult to determine the exact features and weights
  - State of the art techniques use a neural network to obtain EVAL, given a historical dataset

Impact of heuristic on alpha-beta pruning

- How to prune the tree when using a heuristic?

```
function ALPHA-BETA-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state, −∞, +∞)
    return move
```

Impact of heuristic on alpha-beta pruning

- How to prune the tree when using a heuristic?

```
function MAX-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-CUTOFF(state, depth) then return game.EVAL(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(sta , a), α, β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move
```

Recursive basecase no
Longer terminal state!

Impact of heuristic on alpha-beta pruning

- How to prune the tree when using a heuristic?

```
function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-CUTOFF(state, depth) then return game.EVAL(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

Recursive basecase no
Longer terminal state!

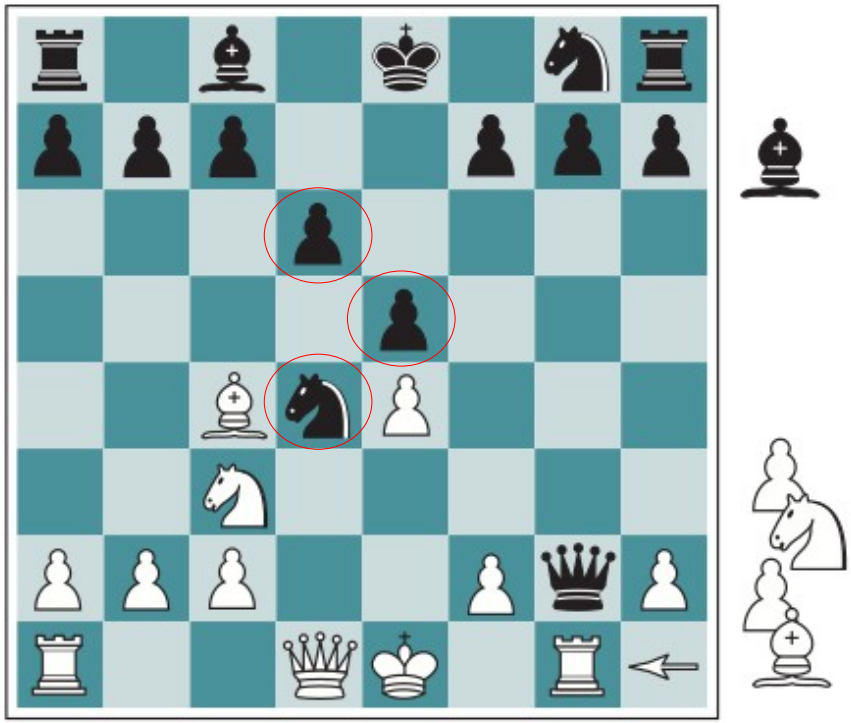
Impact of heuristic on alpha-beta pruning

- There are dangers to pruning now
- Suppose you make use of iterative deepening…
  - The depth limit keeps increasing
- Also, suppose that EVAL is based on the ff. feature
  - e.g., material advantage in chess

Impact of heuristic on alpha-beta pruning

- There are dangers to pruning now
- Suppose you make use of iterative deepening…
  - The depth limit keeps increasing
- Also, suppose that EVAL is based on the ff. feature
  - ial advantage
- At depth limit, black has ex pawns

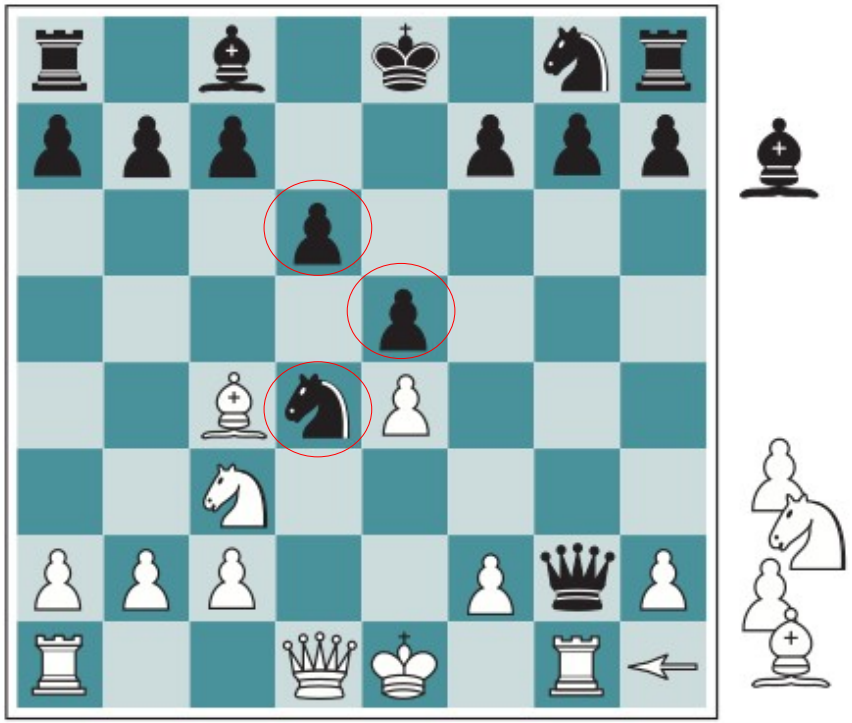
Is the material advantage Sufficient to give a clear picture?



Impact of heuristic on alpha-beta pruning

- There are dangers to pruning now
- Suppose you make use of iterative deepening…
  - The depth limit keeps increasing
- Also, suppose that EVAL is based on the ff. feature
  - e.g., material advant
- At depth limit, black ha pawns

White's turn to move.
Focus on rook.

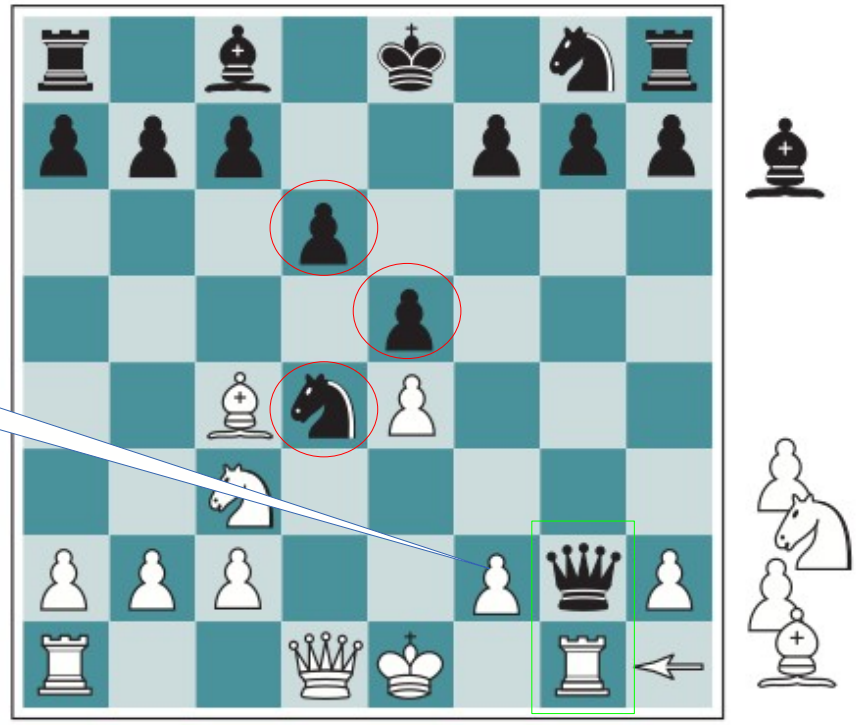


Impact of heuristic on alpha-beta pruning

- You can make problem specific changes
- e.g., EVAL only applies to positions with no pending move
- IS-CUTOFF returns false in prev. state!
- No catch-all rule

Black has no response.
e.g., Lose Queen (9 points)

White's turn to move.
Focus on rook.



Reading

- Sections 5.2.3 – 5.3.2

